

Read this first. Shader Protocol is an early-stage, open-source research project. It runs on developer networks (devnets/testnets) only. The code has **not** been audited. Its zero-knowledge proving keys come from a **single-party development trusted setup that is not secure for real value**. A multi-party ceremony has **not** been started. Parts of the system (adapters other than the first EVM/Solana paths, live message transports, on-chain allowance verification) are incomplete. There is no token, no token sale, no mainnet deployment and no partnership with any chain, exchange or broker named here. Nothing here is financial, legal or tax advice.

01 // THE PROBLEM

PUBLIC LEDGERS LEAK TOO MUCH.

Public blockchains give verifiability. The price is that balances, counterparties, order flow and strategy are visible to everyone, forever.

- **Balances are public.** Wallet identity links to every inflow and outflow.
- **Orders are public before they execute.** Pending trades in public mempools invite copy and sandwich flow.
- **Front-running and MEV follow.** Extractable value comes from reading intent before settlement.
- **Counterparties and strategies leak.** DAOs, desks and agents cannot keep commercial detail private by default.

Centralized venues and naive mixers are unsatisfying. Shader Protocol aims for private-by-default execution with optional, deliberate disclosure.

02 // OVERVIEW

THE IDEA IN ONE PARAGRAPH.

Shader Protocol is a **shielded execution layer**. Users hold value as encrypted notes inside a shielded pool, move it with **Groth16 / Circom** proofs that show a transaction is valid without revealing who, what or how much, and can trade through a **dark pool** where orders stay sealed until match and settlement. Notes carry an **asset id** and a **chainHome** so a note minted on one chain does not open on another unless a teleport mints it there. On top sit selective-disclosure tools (viewing keys, scoped bundles, issuer-signed credentials, regulator capabilities), agent **allowance** notes, private **payroll** batches, and ERC-20 deposits on the EVM pool. The core is chain-agnostic; thin adapters target listed testnets only.

03 // NOTES V2

COMMITMENTS THAT KNOW THEIR HOME.

Devnet note format. Not audited. Not for mainnet. A note opening is:

FIELD	MEANING
value	64-bit amount
blinding, rho	Secret field elements. Deposit rho is chosen by the depositor. Spend outputs derive rho from the input nullifier.
owner	Poseidon(owner domain, spending key)
assetId	Asset identifier
chainHome	Chain that minted the note

Asset and chain are domain-tagged, then packed, so every chain uses the same 6-input commitment:

```
assetTag   = Poseidon(7, assetId)
chainTag   = Poseidon(9, chainHome)
packed     = Poseidon(assetTag, chainTag)
commitment = Poseidon(1, value, blinding, rho, owner, packed)
```

A note minted on Robinhood Chain testnet (46630) does not open on Elysium testnet (99801) unless a teleport mint inserts that commitment into the destination tree.

Nullifier binding. The nullifier is bound to the spending key, the commitment and the leaf index:

```
nullifier = Poseidon(2, spendingKey, commitment, leafIndex)
rhoOut    = Poseidon(6, nullifier, outputIndex)
```

Two notes that reuse rho no longer share a nullifier. Output rhos are derived; a spender cannot choose them freely.

extDataHash binding. Recipient, relayer, fee, pool and chain are bound into the public input so a proof for one pool cannot authorize a payout from another:

```
binder      = Poseidon(5, poolLo, poolHi, chainDomain, withdrawAmount, relayerFee)
extDataHash = Poseidon(recipientLo, recipientHi, relayerLo, relayerHi, binder, 0)
```

Shared circuit family (v2 commitment + same extDataHash): `shielded_transfer.circom` (1-in / 2-out withdraw), `transfer.circom` (2-in / 2-out), `teleport_out.circom` (burn one note; output commitment uses destChain as chainHome). Default `npm run circuit:setup` builds the withdraw circuit only. Transfer and teleport proving keys are not produced by that default run. The setup remains a single-party development setup.

04 // TELEPORT

TWO TRANSPORTS. PER-EPOCH CAPS.

Cross-chain move of shielded value is a burn on the source tree plus an attested mint on the destination. The destination pool's `mintAttested` path accepts a commitment only after a message verifier confirms the payload.

Two-transport agreement. `TwoOfTwoVerifier` requires two independent transports to submit the same `(destChain, commitment, nullifier, amount)` body. A single transport is rejected. Hyperlane and LayerZero adapters in the repository are stubs marked **NOT WIRED**; tests use mock transports. The first designed lane is Robinhood Chain testnet (46630) to Kinetiq Elysium testnet (99801).

Per-epoch cap. Attestation consumes an epoch value budget. When `epochUsed + amount` would exceed `epochCap`, attestation reverts. Epoch boundaries follow `block.timestamp / epochLength`.

Canton is separate: it is **not** a Groth16 verifier. Daml has no BN254 pairing. Canton records a result only when two named attestors authorize it (2-of-N over the named set). If two attestors sign a result that never happened on the EVM pool, Canton records it. That is attestation, not proof verification. The TypeScript Canton adapter still throws `NotImplementedError`. Nothing is deployed to Canton DevNet from this repository.

05 // APPLICATIONS

ALLOWANCES, PAYROLL, ERC-20.

Agent allowance notes. `circuits/allowance_spend.circom` and `src/core/agent/allowance.ts` bind a spend cap and expiry into the note commitment. The circuit checks `amount ⩽ cap`, `amount ⩽ value` and `now ⩽ expiry` with 64-bit range checks. An x402-style helper builds a public payment object; it does not call a live x402 endpoint. **The shielded pool does not yet verify allowance proofs on-chain.** Witness tests cover over-cap and expired `now`.

Private payroll. `src/core/payroll/batch.ts` builds one stealth outgoing note per payee. `chainHome` is inside each commitment, so a payroll for one chain does not open on another.

ERC-20 support. The EVM pool can bind a mock or real ERC-20, accept `depositToken`, and pay recipient and relayer in that token on `shieldedTransfer`. The forge test `testErc20DepositAndWithdrawPaysBothParties` covers that path. Native and token modes are mutually exclusive per pool instance.

06 // MULTI-CHAIN

ONE CORE. THIN ADAPTERS.

Privacy logic lives in a chain-agnostic core. Each chain uses a thin adapter to verify proofs and custody assets. Listing a chain is an engineering target only. **Shader Protocol has no partnership, endorsement or affiliation with Solana, NEAR, Canton / Digital Asset, Robinhood, Kinetiq, Hyperliquid, Base or any other organisation.** Deploy scripts refuse mainnet by chain id or genesis hash. `website/deployments.json` ships with empty addresses. Nothing in this repository has been broadcast.

CHAIN	CHAINHOME / ID	STATUS IN REPO
Solana devnet	1001	Program + LiteSVM tests. Not deployed.
NEAR testnet	1002	near-sdk contract + cargo tests. Not deployed. Open: wee_alloc via near-sdk.
Canton	attestation	2-of-N Daml AttestorSet (SDK 3.6.4). Not a verifier. Not on DevNet.
Robinhood Chain testnet	46630	Same EVM pool bytecode. Not deployed.
Kinetiq Elysium testnet	99801	Arbitrum Orbit settling toward HyperEVM. defineChain in adapter (not viem elysiumTestnet 1338). Not deployed.
HyperEVM testnet	998	Deploy script + pool target. Not deployed.
Base Sepolia	84532	Deploy script + pool target. Not deployed.

> Anvil (31337) is local only. Robinhood mainnet (4663), Solana mainnet genesis and NEAR `mainnet` are rejected by deploy scripts.

07 // DISCLOSURE

PRIVATE BY DEFAULT. DISCLOSE BY CHOICE.

Selective-disclosure features are **tools**. Using them does not make any deployment or user compliant with any law. No regulator has reviewed or approved Shader Protocol.

- **Viewing keys** decrypt notes for an address but cannot spend. Sharing a full viewing key is irreversible for past and future notes until the user moves address.
- **Scoped disclosure bundles** (documented) limit reveal to a time range, asset or note set without handing over the full viewing key.
- **Regulator capabilities** are issuer-signed encryption keys with a scope. Notes sealed to a capability use scope-bound AES-GCM associated data. Expired or re-scoped capabilities fail.
- **Credentials** are Poseidon-hashed field records with an issuer Ed25519 signature over the hash. Presentations must match the issuer store. Self-asserted KYC levels are ignored.
- **No ZK disclosure proof yet.** `generateGroth16KycProof()` throws `NotImplementedError`. There is no Groth16 selective-disclosure KYC circuit in the devnet artifact set. Disclosure receipts today are signature- and key-based, not zero-knowledge proofs.
- **Sanctions screening** is an interface. `DevnetPermissiveScreen` does not check a list.

08 // CEREMONY & KEYS

CEREMONY PLANNED. NOT YET RUN.

Keys from `npm run circuit:setup` are a single-party development setup. Toxic waste is not destroyed. Do not deploy those keys. Anyone who keeps the intermediate parameters can forge proofs.

Ceremony tooling lives under `scripts/ceremony/`. It reuses the pinned Hermez Powers of Tau file `powersOfTau28_hez_final_16.ptau`. Scripts refuse to run unless `SHADE_CEREMONY_ACK=circuits-not-frozen` is set. That flag means you are exercising tooling. It does not open a ceremony. The browser page `website/ceremony.html` is labelled "Ceremony not started" and does not accept contributions. No transcript is published under `circuits/ceremony/published/`.

A real ceremony needs frozen circuit source, a large enough ptau, at least one honest contributor who destroys entropy, and a beacon. None of that has started.

09 // SECURITY TOOLING

SCANNERS HELP. THEY ARE NOT AN AUDIT.

CI runs pinned scanners. Results below reflect the repository's documented tooling status. Tooling is not a substitute for an independent audit.

AREA	RESULT (AS DOCUMENTED)
Foundry unit + invariant	Pool invariants and unit tests in CI (tracked deposits vs balance, known root, garbage proofs must not move funds).
Medusa (EVM)	Local campaign passed property_balance, property_root_known and deposit assertion (445 calls, 0 failures). Does not fuzz Groth16 witnesses.
cargo-fuzz	parse_field and proof_input: local run finished with no crashes (millions of executions).
circomspect	Fails on warnings; CS0010 alone allowlisted.
Slither / Aderyn	Pinned; accepted findings documented in docs/security-tooling.md.
Semgrep / OSV / npm audit	Local rules clean; high npm audit passes; known ignored advisories listed.
Open: wee_alloc	near-sdk 5.11 depends on wee_alloc 0.4.5 (critical advisory). NEAR stays testnet-only. Removing it means forking the SDK.
Open: Picus	Under-constraint job is continue-on-error. Timeout or unknown is a log, not a safety proof. Pending a conclusive safe result.

Tree depth is 20 (frontier). Field inputs are canonical-checked. Dark-pool operators who decrypt order openings learn escrow spending keys; do not custody third-party funds that way.

10 // TOKEN (DESIGN ONLY)

UTILITY BONDING. NO SALE.

There is **no token contract** in this repository. `docs/token-design.md` is a design note, not a launch, not legal advice, and not an offer of a security.

- If ever built: a utility work token to bond relayers and message attestors, slashable on proven fault.
- Fee discounts via circuit are an idea only; that circuit does not exist yet.
- **No revenue share.** Holding would not be a claim on protocol fees.
- **No token sale** is described or authorised by this document or repository.

11 // ROADMAP

TESTNET FIRST. NO DATES PROMISED.

PHASE	MILESTONE	MEANING
0	**Devnet / testnet (now)**	Circuits, multi-chain adapters, website, security CI. Development keys only. Empty deployments.json. Breaking changes expected.
1	**Funded testnet deploys**	Broadcast only with `SHADE_BROADCAST=1` and funded keys. Fill deployments.json. Still unaudited.
2	**Hardening**	On-chain allowance verification, wired transports (not stubs), Picus closure, NEAR allocator path, disclosure ZK if designed.
3	**Independent audit**	Third-party review of circuits, cryptography, contracts and SDK. Findings published. Tooling is not this phase.
4	**Multi-party ceremony**	Only after circuits are frozen. Published transcripts. Replace dev keys.
5	**Guarded mainnet (conditional)**	Only after audit + ceremony, with caps and staged rollout. Not promised.

Each phase starts only when the previous one is genuinely finished.

12 // RISKS

RISKS & DISCLAIMERS.

- **Experimental software.** Shader Protocol is unaudited research code. Bugs may cause total loss of funds or privacy. Use only on devnets/testnets with valueless test tokens.
- **No financial, legal or tax advice.** Nothing here is an offer, solicitation or recommendation. There is no token sale, no yield and no expected return.

- **Not a compliance guarantee.** Selective-disclosure features are tools. No regulator has reviewed or approved Shader Protocol.
- **Development proving keys.** Single-party setup. Forged proofs are possible for anyone who sees intermediate parameters.
- **Incomplete adapters.** Message transports are not wired. Allowance is not verified on-chain. Canton adapter throws. Deployments are empty.
- **Operator-visible dark pool.** The current matcher learns escrow spending keys.
- **Licensing.** circomlib and snarkjs are GPL-3.0. Original repository code is MIT. Confirm that combination before distributing a combined work.

13 // GET INVOLVED

READ THE CODE. BREAK IT. TELL US.

Shader Protocol is open source and testnet-only. Review the circuits and contracts, run the SDK on devnet, and report what you find. Use GitHub private vulnerability reporting for unfixed security bugs.

GITHUB REPOSITORY | SECURITY TOOLING | NOTE FORMAT V2

Shader Protocol Litepaper v2 (draft) · November 2026 · Unaudited · No mainnet · No token sale · No partnerships claimed